



Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251176
Nama Lengkap	David Alexsi Cahyadi
Minggu ke / Materi	11 / Tipe Data List

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Pada bagian ini, tuliskan kembali semua materi yang telah anda pelajari minggu ini. Sesuaikan penjelasan anda dengan urutan materi yang telah diberikan di saat praktikum. Penjelasan anda harus dilengkapi dengan contoh, gambar/ilustrasi, contoh program (source code) dan outputnya. Idealnya sekitar 5-6 halaman.

Sifat-sifat List

List pada Python adalah rangkaian nilai-nilai yang dapat diakses menggunakan satu nama tunggal. Berbeda dengan string yang hanya dapat menyimpan rangkaian karakter, list dapat berisi berbagai tipe data sekaligus, seperti karakter, integer, float, boolean, bahkan list lain di dalamnya. Rangkaian nilai tersebut dituliskan di dalam tanda kurung siku [], misalnya `nilai_ujian = [80,75,70,90]` atau `nama_pahlawan = ['Sukarno', 'Diponegoro']`.

```
nilai_ujian = [80,75,70,90,81,84,92,71,65,80,70]
nama_pahlawan = ['Sukarno', 'Diponegoro', 'Jend. Sudirman', 'Cut Nya Dhien']
nilai_campuran = ['Javascript', 20, 34.4, True]
list_dalam_list = [23, [22, 20], 45]
```

Salah satu perbedaan mendasar antara list dan string terletak pada sifat mutabilitynya. List bersifat **mutable**, artinya nilai di dalamnya dapat diubah secara langsung menggunakan indeks. Sebaliknya, string bersifat **immutable**, sehingga ketika kita mencoba mengubah karakter tertentu di dalam sebuah string, Python akan langsung menampilkan **TypeError: 'str' object does not support item assignment**. Perbedaan lainnya menyangkut identitas objek di memori. Apabila dua variabel string memiliki isi yang sama, Python mengarahkan keduanya ke objek yang sama. Namun pada list, dua list dengan isi yang sama tetap dianggap sebagai dua objek yang berbeda dan terpisah di memori.

```
# definisikan sebuah string
nama = 'Antonius Rachmat'
# ubah karakter index ke-0 menjadi Z
nama[0] = 'Z'
# tidak akan mencapai baris ini karena muncul error
# TypeError: 'str' object does not support item assignment
print(nama)
```

Mengakses dan Mengubah isi List

Setiap elemen dalam list dapat diakses menggunakan indeks, yaitu angka yang menunjukkan posisi elemen di dalam list. Indeks pada Python dimulai dari **0** untuk elemen pertama. Python juga mendukung indeks negatif, di mana **-1** merujuk pada elemen terakhir, **-2** pada elemen kedua dari belakang, dan seterusnya. Selain mengakses elemen satu per satu, kita juga dapat mengambil sebagian isi list menggunakan teknik slicing dengan menuliskan rentang indeks **[start:end]**, di mana nilai **end** bersifat eksklusif sehingga indeks tersebut tidak ikut diambil.

Untuk mengubah isi list, cukup gunakan indeks atau slicing pada sisi kiri pernyataan assignment. Misalnya, **thislist[1] = "jeruk"** akan mengganti elemen pada indeks ke-1 dengan nilai **"jeruk"**. Kita juga dapat mengganti beberapa elemen sekaligus menggunakan slicing, seperti **thislist[1:3] = ["blackcurrant", "watermelon"]** yang akan mengganti elemen indeks 1 dan 2 dengan dua nilai baru tersebut. Kemampuan ini kembali membuktikan sifat mutable yang dimiliki list, sesuatu yang tidak dapat dilakukan pada tipe data string.

```

thislist = ["apple", "banana", "cherry"]

print(thislist[1])
print(thislist[-1])
print(thislist[2:5])
print(thislist[:4])

thislist[1] = "jeruk"

print(thislist)

thislist2 = ["apple", "banana", "cherry", "orange", "kiwi", "mango"]
thislist2[1:3] = ["blackcurrant", "watermelon"]
print(thislist2)

```

Fungsi-fungsi Untuk List

Python menyediakan berbagai cara untuk memanipulasi list, mulai dari operator dasar hingga method bawaan. Operator **+** digunakan untuk menggabungkan dua list menjadi satu list baru, sedangkan operator ***** digunakan untuk mengulangi isi list sebanyak n kali. Selain operator, Python juga menyediakan sejumlah method yang dapat dipanggil langsung dari objek list. Method **append()** digunakan untuk menambahkan elemen baru di akhir list sebagai satu kesatuan objek, sedangkan **extend()** menambahkan elemen-elemen baru secara individual. Method **sort()** digunakan untuk mengurutkan elemen list, **reverse()** untuk membalik urutan elemen, **pop()** untuk menghapus elemen berdasarkan indeks sekaligus mengembalikan nilainya, **del** untuk menghapus elemen tanpa mengembalikan nilai, dan **remove()** untuk menghapus elemen berdasarkan nilai tertentu.

1. **append.** Metode yang digunakan untuk menambahkan elemen baru dan dianggap sebagai kesatuan objek pada bagian akhir list.

```

>>> nama = ['kuncoro', 'felix', 'ryan', 'yuan']
>>> nama.append(['tejo', 'ujo'])

```

2. **extend.** Metode yang digunakan untuk menambahkan elemen pada sebuah list, dan melakukan setiap element baru sebagai elemen list secara individual.

```

>>> nama = ['kuncoro', 'felix', 'ryan', 'yuan']
>>> nama.extend(['tejo', 'ujo'])

```

3. **sort.** Metode yang digunakan untuk mengurutkan elemen pada sebuah list.

```

>>> nama = ['kuncoro', 'felix', 'ryan', 'yuan']
>>> nama.sort()

```

4. **pop.** Metode yang digunakan untuk menghapus elemen pada sebuah list. Jika indeks elemen sudah diketahui dan ingin mendapatkan nilai yang dihapus.

```
>>> nama = ['kuncoro', 'felix', 'ryan', 'yuan']
>>> nama.pop(1) #mengembalikan 'felix'
```

5. **del**. Metode untuk menghapus elemen list tapi tidak perlu menampilkan elemen yang dihapus.

```
>>> nama = ['kuncoro', 'felix', 'ryan', 'yuan']
>>> del nama[2]
```

6. **remove**. Metode untuk menghapus elemen list berdasarkan nilai tertentu.

```
>>> nama = ['kuncoro', 'felix', 'ryan', 'yuan']
>>> nama.remove('kuncoro')
```

7. **reverse**. Metode untuk membalik urutan list.

```
>>> nama = ['kuncoro', 'felix', 'ryan', 'yuan']
>>> nama.reverse()
```

Selain method tersebut, Python juga menyediakan fungsi built-in yang dapat digunakan pada list, yaitu **min()** untuk mencari nilai terkecil, **max()** untuk mencari nilai terbesar, **sum()** untuk menjumlahkan seluruh elemen, dan **len()** untuk mengetahui jumlah elemen dalam list. Python juga mengenal konsep list comprehension, yaitu cara penulisan yang lebih ringkas untuk membuat list baru berdasarkan list yang sudah ada, dengan format **[ekspresi for item in list if kondisi]**. Sebagai contoh, untuk mengambil semua elemen yang mengandung substring **“or”** dari sebuah list nama, cukup ditulis **[x for x in nama if “or” in x]**. List comprehension membuat kode menjadi lebih singkat, mudah dibaca, dan tetap mudah dipahami.

1. **min()**: untuk mengetahui nilai minimal dari elemen list (angka). Contoh:

```
>>> nama = [5,3,90,1]
>>> min(nama)
```

2. **max()**: untuk mengetahui nilai maksimal dari elemen list (angka). Contoh:

```
>>> nama = [5,3,90,1]
>>> max(nama)
```

3. **sum()**: untuk mengetahui jumlah dari total elemen list (angka). Contoh:

```
>>> nama = [5,3,90,1]
>>> sum(nama)
```

4. **len()**: untuk mengetahui panjang list. Contoh:

```
>>> nama = [5,3,90,1]
>>> len(nama)
```

```
nama = ["anton", "kuncoro", "buntoro", "juworo", "margono"]
newlist = [x for x in nama if "or" in x]
```

Contoh untuk mengubah semua elemen list menjadi huruf besar.

```
nama = ["anton", "kuncoro", "buntoro", "juworo", "margono"]
newlist = [x.upper() for x in nama]
```

Contoh untuk memeriksa mengambil elemen list yang panjangnya lebih dari 6 huruf.

```
nama = ["anton", "kuncoro", "buntoro", "juworo", "margono"]
newlist = [x for x in nama if len(x) > 6]
```

List Sebagai Parameter fungsi

Ketika sebuah list digunakan sebagai parameter fungsi, list tersebut tidak disalin melainkan dikirimkan sebagai referensi ke objek aslinya. Hal ini berarti perubahan yang dilakukan terhadap list di dalam fungsi akan langsung memengaruhi nilai list di luar fungsi. Sifat ini merupakan konsekuensi langsung dari sifat mutable yang dimiliki list. Sebagai contoh, jika sebuah fungsi mengubah elemen pada indeks ke-0 menjadi **"anton"**, maka list asli yang dikirimkan sebagai argumen pun akan ikut berubah tanpa perlu mengembalikan nilai apapun dari fungsi tersebut.

Namun perlu diperhatikan, jika di dalam fungsi kita menggunakan operasi slicing seperti **data[1:]**, operasi tersebut tidak mengubah list asli melainkan menghasilkan list baru. Untuk memperbarui list asli, nilai kembalian dari fungsi tersebut harus ditangkap dan disimpan kembali ke variabel yang sama. Penggunaan list sebagai parameter fungsi membawa sejumlah keuntungan, di antaranya fungsi menjadi lebih fleksibel karena dapat memproses berbagai jenis data, lebih reusable karena dapat dipanggil dengan list yang berbeda-beda, serta membantu dalam pembagian tugas dan pengorganisasian data sebelum diproses lebih lanjut oleh fungsi.

BAGIAN 2: LATIHAN MANDIRI (60%)

Pada bagian ini anda menuliskan jawaban dari soal-soal Latihan Mandiri yang ada di modul praktikum. Jawaban anda harus disertai dengan source code, penjelasan dan screenshot output.

Github: https://github.com/DavidCahyadi1013/71251176_David-Alexsi-Cahyadi.git

Latihan 10.1

Soal:

Latihan 10.1 Buatlah fungsi untuk mencari 3 bilangan terbaik dari sebuah list. Bilangan terbaik adalah bilangan dimulai dari yang paling tinggi.

Source Code:

```
def terbaik(data):  
    k = list(set(data))  
    k.sort(reverse=True)  
    return k[:3]  
  
angka = []  
n = int(input("Input jumlah bilangan: "))  
for i in range(n):  
    bil = int(input(f"Masukkan bilangan ke-{i+1}: "))  
    angka.append(bil)  
  
akhir = terbaik(angka)  
print("3 bilangan terbaik:",akhir)
```

Output:

```
PS D:\PrAlpro\Pertemuan 11> & C:/Users/dvdal/AppData/Local/Programs/Python/Python313/python.exe "d:/PrAlpro/Pertemuan 11/soal01"  
Input jumlah bilangan: 5  
Masukkan bilangan ke-1: 32  
Masukkan bilangan ke-2: 23  
Masukkan bilangan ke-3: 43  
Masukkan bilangan ke-4: 53  
Masukkan bilangan ke-5: 11  
3 bilangan terbaik: [53, 43, 32]
```

Penjelasan:

Program dimulai dengan mendefinisikan fungsi **terbaik(data)** yang menerima sebuah list sebagai parameter. Di dalam fungsi tersebut, list yang diterima terlebih dahulu dikonversi menggunakan **set()** untuk menghapus nilai-nilai duplikat, kemudian dikembalikan menjadi list menggunakan **list()**. Setelah itu, list diurutkan dari nilai terbesar ke terkecil menggunakan method **sort(reverse=True)**. Terakhir, diambil 3 elemen pertama menggunakan slicing **k[:3]** karena setelah diurutkan secara descending, 3 elemen pertama sudah pasti merupakan 3 bilangan terbesar, dan nilai tersebut dikembalikan sebagai hasil fungsi.

Pada bagian utama program, disiapkan terlebih dahulu sebuah list kosong **angka = []** yang nantinya akan diisi oleh input pengguna. Pengguna diminta memasukkan jumlah bilangan yang ingin diproses melalui variabel **n**. Program kemudian melakukan perulangan sebanyak **n** kali, di mana setiap iterasinya meminta satu bilangan dari pengguna dan langsung menambahkannya ke dalam list **angka** menggunakan method **append()**. Setelah semua bilangan terkumpul, list **angka** dikirimkan ke fungsi **terbaik()** sebagai argumen, hasilnya ditampung dalam variabel **akhir**, dan kemudian dicetak ke layar sebagai 3 bilangan terbaik.

Latihan 10.2

Soal:

Latihan 10.2 Buatlah program yang dapat menerima input bilangan-bilangan dari pengguna dan kemudian jika pengguna memasukan kata "done" maka program akan menampilkan bilangan rata-ratanya.

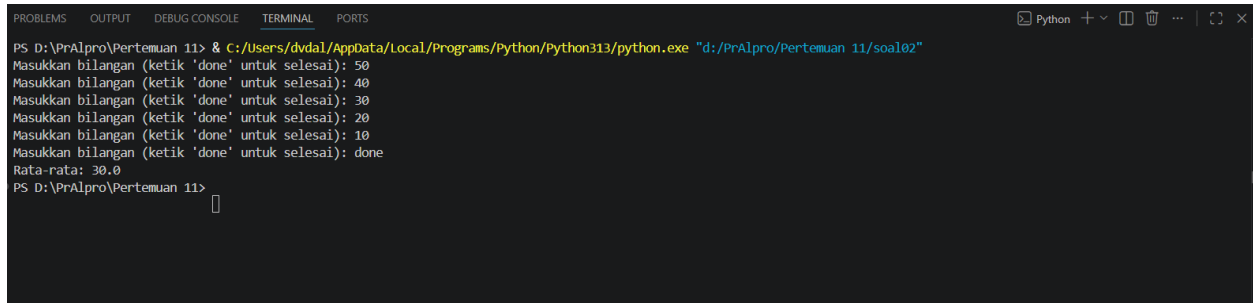
Source Code:

```
angka = []

while True:
    sok = input("Masukkan bilangan (ketik 'done' untuk selesai): ")
    if sok == "done":
        break
    angka.append(int(sok))
```

```
rata = sum(angka)/len(angka)
print("Rata-rata:",rata)
```

Output:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS D:\PrAlpro\Pertemuan 11> & C:/Users/dvdal/AppData/Local/Programs/Python/Python313/python.exe "d:/PrAlpro/Pertemuan 11/soal02"
Masukkan bilangan (ketik 'done' untuk selesai): 50
Masukkan bilangan (ketik 'done' untuk selesai): 40
Masukkan bilangan (ketik 'done' untuk selesai): 30
Masukkan bilangan (ketik 'done' untuk selesai): 20
Masukkan bilangan (ketik 'done' untuk selesai): 10
Masukkan bilangan (ketik 'done' untuk selesai): done
Rata-rata: 30.0
PS D:\PrAlpro\Pertemuan 11>
```

Penjelasan:

Program dimulai dengan menyiapkan sebuah list kosong **angka = []** yang nantinya akan menampung seluruh bilangan yang dimasukkan oleh pengguna. Kemudian program memasuki perulangan **while True** yang akan terus berjalan tanpa henti sampai ada kondisi yang menghentikannya.

Di dalam perulangan, program meminta input dari pengguna dan menyimpannya ke variabel **sok**. Setiap kali pengguna memasukkan sesuatu, program memeriksa apakah isinya adalah kata **“done”**. Jika ya, perulangan dihentikan menggunakan **break**. Jika bukan, input tersebut dikonversi menjadi integer menggunakan **int()** dan langsung ditambahkan ke dalam list **angka** menggunakan method **append()**.

Setelah perulangan berhenti, program menghitung rata-rata dengan membagi jumlah seluruh elemen list menggunakan **sum(angka)** dengan banyak elemen menggunakan **len(angka)**, dan hasilnya disimpan ke variabel **rata**. Nilai rata-rata tersebut kemudian dicetak ke layar.

Latihan 10.3

Soal:

Latihan 10.3 Buatlah program yang menerima sebuah file teks yang berisi sebuah artikel atau berita. Kemudian program akan memeriksa semua kata yang ada di dalam artikel teks tersebut, kemudian program akan menampilkan semua kata yang unik dari artikel tersebut.

Source Code:

```
import re
def ku (file):
    with open(file, "r", encoding="utf-8") as f:
        isi = f.read()

        semua = re.findall(r'\b\w+\b', isi.lower())
        ku = list(dict.fromkeys(semua))
        return isi, ku

file = input ("Masukkan nama file artikel/berita (.txt): ")
isi, hasil = ku(file)

print(f"\n===== ISI BERITA '{file}' =====")
print(isi)
print(f"\n===== KATA UNIK =====")
print(hasil)
print(f"\nTotal kata unik: {len(hasil)}")
```

Output:

[illegible]

Penjelasan:

Program dimulai dengan mengimpor library **re** yang digunakan untuk operasi regex. Kemudian didefinisikan fungsi **ku(file)** yang menerima nama file sebagai parameter. Di dalam fungsi tersebut, file teks dibuka menggunakan **open()** dengan mode baca “r” dan encoding “utf-8”, kemudian seluruh isinya dibaca menggunakan **f.read()** dan disimpan ke variabel **isi**.

Setelah isi file terbaca, program menggunakan **re.findall(r'\b\w+\b', isi.lower())** untuk mencari semua kata dalam teks. Teks terlebih dahulu diubah menjadi huruf kecil menggunakan **.lower()** agar kata yang sama namun berbeda kapitalisasi tidak dianggap berbeda. Pola regex **\b\w+\b** bekerja dengan mencocokkan rangkaian karakter huruf atau angka yang dibatasi oleh batas kata, sehingga tanda baca dan simbol otomatis tidak ikut terambil. Hasilnya disimpan ke variabel **semua**.

Selanjutnya, **semua** dikonversi menggunakan **dict.fromkeys()** yang berfungsi menghapus kata-kata duplikat sekaligus mempertahankan urutan kemunculan kata sesuai urutan di dalam berita. Hasilnya kemudian dikonversi menjadi list menggunakan **list()** dan disimpan ke variabel **ku**, lalu dikembalikan bersama **isi** sebagai dua nilai kembalian fungsi.

Pada bagian utama program, pengguna diminta memasukkan nama file berita melalui **input()**. Nama file tersebut dikirimkan ke fungsi **ku()** sebagai argumen, dan hasilnya ditangkap oleh dua variabel sekaligus yaitu **isi** dan **hasil**. Kemudian program mencetak isi berita secara lengkap, diikuti daftar kata unik dan total jumlah kata unik yang ditemukan menggunakan **len(hasil)**.